Ders 8 Genişletilmiş Notu: Destek Vektör Makineleri: Margin, Soft Margin, Kernel ve Çok Sınıflı SVM

Ders 8 Genişletilmiş Notu: Destek Vektör Makineleri: Margin, Soft Margin, Kernel ve Çok Sınıflı SVM

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Perceptron Hatırlatması
- 2. Perceptron'un Sorunları
- 3. Karar Sınırının Yeri Neden Önemli?
- 4. Destek Vektörleri: Ön Cephedeki Örnekler
- 5. SVM'in Kısa Tarihçesi
- 6. Margin
- 7. Margin'in Matematiği
- 8. Optimizasyon Problemi ve Kısıtlar
- 9. Lagrange Çarpanları ve Çözümün Yapısı
- 10. Bilinmeyen Bir Örneğin Sınıflandırılması
- 11. Sınıra Yakın Aykırı Örnekler ve Soft Margin
- 12. Doğrusal Ayrılamayan Veri ve Kernel Fonksiyonları
- 13. Kernel ve C Seçimi: Doğrulama ile Model Seçmek
- 14. Çok Sınıflı SVM: One-Against-One
- 15. Çok Sınıflı SVM: One-Against-All
- 16. Hangisi Daha Başarılı?

1. Perceptron Hatırlatması

Hoca derse, destek vektör makinelerinde (Support Vector Machines, SVM) benzer bir yapı kullanılacağı için geçen haftanın konusu olan perceptron'u uzun uzun hatırlatarak başladı. Geçen derse katılmayanların bu kısmı bilmeden SVM'i takip etmekte zorlanacağını özellikle söyledi.

Elimizde n özelliği olan bir giriş örneği olsun: $\mathbf{x} = (x_1, x_2, \dots, x_n)$. Doğrusal olarak ayrılabilen iki sınıfımız varsa, gerçek çıkış şöyle hesaplanır:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + b)$$

Burada $\mathbf{w}^T\mathbf{x}$ bir iç çarpımdır (dot product):

$$\mathbf{w}^T\mathbf{x} = w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

Hoca derste ağırlıkları v harfiyle yazdı ve konuştu ("vx artı b"); bu notta slaytlardaki ve kısa nottaki yaygın gösterimle $\mathbf{w}$ kullanılıyor. b terimi bias'tır. Perceptron'da bunu $x_0 = 1$ girişine bağlı w_0 ağırlığı ya da eşik seviyesi olarak görmüştük. Görevi hiper düzlemin (hyperplane) orijine göre konumunu ayarlamaktır.

Hiper düzlem, iki sınıfı birbirinden ayıran karar sınırıdır. Hoca şunları hatırlattı:

- **w** ağırlık vektörü hiper düzleme diktir.
- Hiper düzlemin kendisi $\mathbf{w}^T\mathbf{x} + b = 0$ olan yerdir.
- Bir tarafı $\mathbf{w}^T\mathbf{x} + b > 0$ olan bölge, diğer tarafı $\mathbf{w}^T\mathbf{x} + b < 0$ olan bölgedir.
- Sign fonksiyonu kullanılırsa çıkış y, $+1$ ya da -1 olur.

Perceptron'un yapısı da aynıdır: $x_1, x_2, \ldots, x_n$ girişleri $w_1, w_2, \ldots, w_n$ ağırlıklarıyla çarpılıp toplanır, buna $x_0 = 1$ girişine bağlı bias (w_0) eklenir ve sonuç aktivasyon fonksiyonundan geçerek çıkışı verir. Perceptron, iki sınıfı doğrusal olarak ayıran en basit yapıdır.

2. Perceptron'un Sorunları

Hoca tek katmanlı perceptron'un bilinen sorunlarını şöyle sıraladı:

- Yavaş öğrenir.
- Özellik sayısı arttığında aşırı öğrenme (overfitting) sorunu vardır. İki, üç, beş özellikte bir dereceye kadar idare eder, ama özellik sayısı büyüdükçe sorun belirginleşir.
- Doğrusal olarak ayrılamayan problemleri çözemez.

Hocaya göre bunlar asıl problem değildir. Çok daha büyük bir problem vardır: perceptron karar sınırını rastgele bir yere koyar. Dersin geri kalanı bu problemin çözümü üzerine kurulu.

3. Karar Sınırının Yeri Neden Önemli?

Hoca iki özellikli (x_1, x_2) bir sistem düşündü. Tahtaya iki sınıfın eğitim örneklerini çizdi ve bu iki sınıfı ayıran birkaç farklı karar sınırı gösterdi. Sonra şunu sordu: "Aradaki karar sınırı en fazla ne kadar olabilir? Kaç farklı karar sınırı çizebilirim?" Bir öğrenci "sonsuz" diye cevap verdi. Doğrusu da budur: İki sınıfın arasına sonsuz sayıda karar sınırı çizilebilir.

Peki bu neden sorun? Hoca sınıflara A ve B adını verdi ve iki sınıfın arasında bir bilinmeyen örnek işaretledi.

- Birinci karar sınırına göre bu örnek hangi sınıfa aittir? Öğrenciler: B.
- Karar sınırı başka bir yere çizilirse aynı örnek hangi sınıfa aittir? Öğrenciler: A.

Aynı örnek, karar sınırının nereye çizildiğine göre farklı sınıfa atanıyor. Buradan çıkan sonuç şudur: Eğitim örneklerini %100 başarıyla sınıflandıran sonsuz sayıda karar sınırı vardır, ama önemli olan test örneklerinin ne kadar başarıyla sınıflandırılacağıdır. Test başarısını da karar sınırının yeri belirler. O halde yapmamız gereken şey, karar sınırını en başarılı sınıflandırmayı sağlayacak yere koymaktır.

Hoca öğrencilere matematik kullanmadan bir fikir sordu: Karar sınırı nereye konursa etkin bir çözüm sağlar? Bir öğrenci "iki sınıfın tam ortasında olması" dedi. Hoca bu cevabı daha doğru bir cümleyle düzeltti:

> **Hocanın vurgusu:** "İki sınıfın tam ortası" demeyelim. Doğru ifade şu: Karar sınırı, iki sınıfın birbirine en yakın örneklerine eşit mesafede olmalıdır.

Sınır bir sınıfa yakın olursa bir sonuç, öbür sınıfa yakın olursa başka bir sonuç alınır. Eğitimde başarı yüksek çıkabilir, ama testte o kadar başarı olmaz. Sınır, iki sınıfın en yakın örneklerine eşit uzaklıktaysa hocanın deyişiyle "eşit şartlarda savaş" sağlanmış olur.

4. Destek Vektörleri: Ön Cephedeki Örnekler

Hoca sınıfların arkasında kalan örnekleri gösterdi. Bu örneklerin sınıflandırmaya pek bir etkisi yoktur: Karar sınırını nereye koyarsanız koyun, arka taraftaki bir örnek her zaman doğru sınıflandırılır. Sınıflandırmayı asıl belirleyen örnekler öndekilerdir. Hoca bunları "ön cephede savaşan, tam kaleyi savunan savaşçı örnekler" diye anlattı.

Destek vektör makinelerinde bu savaşçı örnekler destek vektörleri (support vectors) olur. Arka plandaki örneklerin hiçbir etkisi olmaz. "Vektör" denmesinin sebebi, her örneğin aslında bir vektör olmasıdır: Bu örnekte iki özellik var, başka bir problemde bir örnek 20 özellikli bir vektör olabilir. Destek vektörleri, sınırı koruyan ve karar sınırının nasıl çizilmesi gerektiğini belirleyen eğitim örnekleridir.

SVM'in yaptığı iş şudur: Karar sınırını eğitim örneklerine göre öyle bir yere koymak ki iki tarafı ayıran mesafe maksimum olsun.

5. SVM'in Kısa Tarihçesi

Hoca "History of SVM" slaytından kısaca bahsetti:

- SVM 1992'de ortaya atılmış bir yaklaşımdır. Hoca "Ruslar tarafından ortaya atılmış" dedi (bkz. Ders Sonrası Notlar).
- Asıl popüler olması 2000'lerde oldu.
- El yazısı karakter tanımada yaklaşık 1,1 (yüzde) test hatası verdi. Bu, o problem için özel tasarlanmış LeNet yapay sinir ağının test hatasıyla aynı seviyedeydi; bu yüzden dikkat çekti.
- Derin öğrenme yöntemleri yaygınlaşana kadar en çok kullanılan ve başarısı en yüksek yöntemlerden biriydi.

SVM'in en önemli özelliği, hem doğrusal ayrılabilen sınıflarda hem de kernel kullanımıyla doğrusal ayrılamayan (non-linear) sınıflarda başarılı sonuçlar vermesidir.

6. Margin

Slayttaki şekilde yine iki sınıf ve aralarına çizilebilecek sonsuz karar sınırı vardır. SVM'in fikri şudur: Sınıra en yakın örneklerin üzerinden geçen, karar sınırına paralel iki düzlem çizilir. Bu iki düzlem arasındaki bölgeye margin denir. Karar sınırı bu margin'in tam ortasına çizilir. Margin'in genişliği M ise karar sınırı her iki tarafa $M/2$ uzaklıktadır.

Üç düzlem şöyledir:

- Karar sınırı: $\mathbf{w}^T\mathbf{x} + b = 0$
- Bir sınıfın sınır düzlemi: $\mathbf{w}^T\mathbf{x} + b = +1$
- Diğer sınıfın sınır düzlemi: $\mathbf{w}^T\mathbf{x} + b = -1$

Karar sınırından $+1$ düzlemine olan uzaklık d_+, -1 düzlemine olan uzaklık d_- ise iki şart vardır:

$$d_+ = d_-, \qquad M = d_+ + d_-$$

Yani artı sınıfına olan mesafe eksi sınıfına olan mesafeye eşit olmalı, margin de bu iki mesafenin toplamı olmalıdır. Amaç, margin'i maksimum yapan hiper düzlemin ağırlıklarını bulmaktır.

7. Margin'in Matematiği

Hoca matematiği "ezberlemeniz gerekmiyor, ama neyin nereden çıktığını bilin" diyerek anlattı.

> **Hocanın vurgusu:** Bu matematiği ezberlemeniz falan gerekmiyor; neyin nereden çıktığını bilin diye anlatıyorum.

Herhangi bir $\mathbf{x}$ noktasının hiper düzleme olan uzaklığı şöyledir:

$$d = \frac{|\mathbf{w}^T\mathbf{x} + b|}{\|\mathbf{w}\|}, \qquad \|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}$$

Destek vektörleri tam margin sınırında durur. Bu yüzden destek vektörleri için $\mathbf{w}^T\mathbf{x} + b$ ya $+1$ ya da -1'dir. Uzaklık formülünde payın mutlak değeri 1 olur ve her destek vektörünün hiper düzleme uzaklığı şu çıkar:

$$d_+ = d_- = \frac{1}{\|\mathbf{w}\|}$$

Margin bu iki uzaklığın toplamıdır:

$$M = d_+ + d_- = \frac{2}{\|\mathbf{w}\|}$$

Hoca "marjin'deki 2 buradan geliyor" dedi. Bu formül python ile örnek bir $\mathbf{w}$ ve b için de kontrol edildi: $+1$ düzlemi üzerindeki bir noktanın karar sınırına uzaklığı $1/\|\mathbf{w}\|$, margin $2/\|\mathbf{w}\|$ çıkıyor.

Sonra hoca şunu sordu: Margin'in maksimum olması için ne olması gerekir? $\|\mathbf{w}\|$ paydada olduğu için ağırlıkların büyüklüğü minimum olmalıdır. Öğrenciler de bunu onayladı.

Hocanın vurgusu: Margin'i maksimize etmek, $\|\mathbf{w}\|$'yi minimize etmek demektir.

Hesap kolaylığı için $\frac{2}{\|\mathbf{w}\|}$'yi maksimize etmek yerine eşdeğer olarak şu ifade minimize edilir:

$$\frac{1}{2}\|\mathbf{w}\|^2$$

8. Optimizasyon Problemi ve Kısıtlar

Margin'i maksimize ederken bir şart da sağlanmalıdır: Bütün eğitim örnekleri margin'in doğru tarafında kalmalıdır. y_i çıkışı $+1$ ya da -1 değerini alır:

- $y_i = +1$ ise: $\mathbf{w}^T\mathbf{x}_i + b \geq 1$
- $y_i = -1$ ise eşitsizlik yön değiştirir: $\mathbf{w}^T\mathbf{x}_i + b \leq -1$

İki ifade y_i ile çarpılarak tek bir kısıta dönüşür:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \quad \text{ya da} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 \geq 0$$

Hoca neden $\mathbf{w}^T\mathbf{x}$ yazıldığını da açıkladı: $\mathbf{w}$ ve $\mathbf{x}$ ikisi de vektördür. Gösterim, $w_1x_1 + w_2x_2 + ...$ iç çarpımını ifade etmek içindir.

8.1 Soru-Cevap: Karar sınırı sıfırken neden ±1 yazıyoruz?

Soru: Karar sınırını neden 1 kabul ettik, anlamadım. $+1$ veya -1 olmasını anlamadım.

Cevap: Karar sınırı 1 değil, sıfırdır: $\mathbf{w}^T\mathbf{x} + b = 0$. Hoca cevabı tahta üzerinden verdi; bu sırada bağlantısı koptu (bilgisayarın şarjı %1'e düşmüştü) ve ders yaklaşık 10 dakika ara verdi. Aradan sonra soruyu baştan cevapladı:

Perceptron'da karar sınırı sıfırdı; büyükler bir tarafta, küçükler öbür taraftaydı. Örneklerin sınıra ne kadar uzak durduğuyla ilgilenmiyorduk. Karar sınırını gelişigüzel bir yere çizip "sıfır gösteriyor, tamamdır" diyorduk. SVM'de ise karar sınırını iki sınıfın sınır örneklerine eşit uzaklıkta olacak bir yere koymak istiyoruz. Bunun için margin'i hesaplamamız gerekir. Bu yüzden artık sınırdaki örneklerin bilgisi önemlidir. Bir sınıfın sınır örnekleri $\mathbf{w}^T\mathbf{x} + b = +1$ düzleminde, diğer sınıfınkiler $\mathbf{w}^T\mathbf{x} + b = -1$ düzlemindedir. Karar sınırı sıfır olmasına rağmen kısıt ≥ 1 şeklinde yazılır, çünkü y_i ya $+1$ ya -1'dir ve sınır örneklerinin çıkışı bu değerleri alır. Öğrenci açıklamadan sonra anladığını söyledi.

9. Lagrange Çarpanları ve Çözümün Yapısı

Artık iki şartı birlikte sağlamamız gerekir:

- $\frac{1}{2}\|\mathbf{w}\|^2$ minimum olmalı (margin maksimum olsun),
- her eğitim örneği için $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$ sağlanmalı.

Hoca bunun Lagrange çarpanları (Lagrange multipliers) ile çözülen bir kısıtlı optimizasyon problemi olduğunu söyledi; problemin bu biçimine dual problem denir. "Matematiği keseceğim" dedi, ama Lagrange fonksiyonunu yine de yazdı. Her eğitim örneği için bir $\alpha_i \geq 0$ çarpanı tanımlanır:

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{N} \alpha_i \left[y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \right]$$

Hoca çözüme girmedi, yalnız kullanılacak sonuçları verdi:

- Çözüm sırasında şu eşitlik sağlanır: $\sum_{i=1}^{N} \alpha_i y_i = 0$.
- Bilinmeyen ağırlık değerleri, Lagrange çarpanları çözüldüğünde şöyle elde edilir:

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i$$

- $\alpha_i > 0$ ise $\mathbf{x}_i$ bir destek vektörüdür; margin üzerindedir.
- $\alpha_i = 0$ ise $\mathbf{x}_i$ destek vektörü değildir; margin üzerinde olmayan, arkada kalan bir örnektir.

Bu son iki madde önemlidir: Ağırlık vektörü yalnız $\alpha_i > 0$ olan örneklerden, yani destek vektörlerinden oluşur. Arka plandaki örneklerin α değeri sıfır olduğu için toplama hiçbir katkıları yoktur.

10. Bilinmeyen Bir Örneğin Sınıflandırılması

Ağırlıklar bulunduktan sonra bilinmeyen bir **z** örneği (test örneği; hoca İngilizce adıyla "query sample" dedi) şöyle sınıflandırılır. **w** yerine yukarıdaki ifadeyi koyarsak:

$$f(\mathbf{z}) = \sum_{i \in SV} \alpha_i y_i \, \mathbf{x}_i^T \mathbf{z} + b$$

Burada $\mathbf{x}_i$'ler eğitim verisindeki destek vektörleridir (SV). Hesap adım adım şöyledir:

1. Her destek vektörü $\mathbf{x}_i$ ile bilinmeyen örnek **z**'nin iç çarpımı alınır.
2. Bu değer o destek vektörünün sınıf etiketi y_i ($+1$ ya da -1) ve α_i ile çarpılır.
3. Bütün destek vektörleri için bulunan değerler toplanır ve b eklenir.
4. Sonuç sıfırdan büyükse örnek bir sınıfa (A), sıfırdan küçükse diğer sınıfa (B) atanır.

Bu haliyle yapılan iş ikili sınıflandırmadır (binary classification). Hocanın örneği: spam olan ve spam olmayan e-postalar.

Hoca slayttaki bir örnek üzerinde, $\mathbf{w}^T\mathbf{x} + b = 1$ (ve -1) düzlemleri üzerindeki örneklerin α değerlerinin sıfırdan farklı, margin üzerinde olmayan örneklerinkinin sıfır olduğunu gösterdi.

Hoca bunun kazancını k en yakın komşu dersine bağladı: Orada da işe yaramayan örneklerin elenmesini görmüştük. SVM'de işe yaramayan örneklerle hiç iş yapılmaz. Bilinmeyen bir örneği sınıflandırırken yalnız margin üzerindeki destek vektörleri kullanılır.

> **Hocanın vurgusu:** Sınıflandırmada yalnız destek vektörleri ($\alpha_i > 0$ olanlar) kullanılır. $\alpha_i = 0$ olan örnekler hesaplamaya girmez.

Hoca bu konuyu çok beğendiğini söyledi: SVM'i tasarlayanlar, yapay sinir ağlarının (çok katmanlı perceptron dahil) sınıflandırmadaki başarısının biraz düşük olma problemini margin'i maksimize ederek çözmüşler.

11. Sınıra Yakın Aykırı Örnekler ve Soft Margin

11.1 Soru-Cevap: Aykırı değerler

Soru: Aykırı değerlerde (outlier) SVM nasıl bir tepki gösteriyor?

Cevap: Aykırı örnek sınıfın arka tarafında, uzakta bir yerdeyse hiçbir etkisi yoktur. Sorun, sınıra yakın, aradaki bölgede ya da ters tarafa kaymış birkaç örnek olduğunda çıkar. Hoca bunların tam aykırı değer olmadığını, "birazcık dışarıda kalan, arada bir iki tane kalan örnekler" olduğunu söyledi ve bunu anlatmaya geçti.

11.2 Sorun

Diyelim ki iki sınıfın arasında, nadir görülen birkaç örnek var. Karar sınırını bu örneklere bakarak da çizebiliriz. Ama bu durumda margin çok daralır. Sınırdaki aykırı örneklere göre margin çizmek margin'i gereksiz yere daraltır ve sınıflandırma başarısını olumsuz etkiler. Arkadaki aykırı örnekler bizi ilgilendirmez; ön cephedeki aykırı örnekler sorun olur.

Çözüm, margin hesabını yumuşatmak, yani daha az hassas hale getirmektir. Buna soft margin (yumuşak margin) denir. Ortaya çıkan hiper düzleme de soft margin hyperplane denir.

11.3 Gevşek değişkenler

Bunun için gevşek değişken (slack variable, "yumuşatıcı değişken") ξ_i kullanılır. Amaç, $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$ şartını sağlamayan örnekleri de toparlayacak şekilde eşitsizliği yeniden yazmaktır:

- $y_i = +1$ (A sınıfı) için: $\mathbf{w}^T\mathbf{x}_i + b \geq 1 - \xi_i$
- $y_i = -1$ (B sınıfı) için: $\mathbf{w}^T\mathbf{x}_i + b \leq -1 + \xi_i$
- Her zaman: $\xi_i \geq 0$

Hoca tahtaya iki özellikli bir örnek çizdi. Çoğu örnek kendi sınıfının tarafındaydı; bir yuvarlak örnek ve bir çarpı örnek ise sınıra yakın ya da ters taraftaydı. Örnekleri üç türe ayırdı:

- **1. tür, margin destek vektörleri (margin support vectors):** Tam margin üzerindeki örnekler. Gevşek değişkenleri sıfırdır ($\xi_i = 0$).
- **2. tür, margin dışı destek vektörleri (non-margin support vectors):** Hocanın deyişiyle "biraz şaşırmış, yolun öbür tarafına kaymış" örnekler. Margin'in içine girmişlerdir.
- **3. tür, hatalar (errors):** Hocanın deyişiyle "iyice yolunu şaşırmış", karar sınırının yanlış tarafına geçmiş örnekler.

11.4 Optimizasyon problemi ve C parametresi

Ağırlık değerleri şu ifadeyi minimize edecek şekilde belirlenir:

$$\min_{\mathbf{w},b,\xi} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i$$

Kısıtlar yukarıdaki gevşek değişkenli eşitsizliklerdir. Burada C düzenleme sabitidir (regularization constant); hoca buna trade-off parametresi de dedi. Önceki yaklaşımdan tek fark, C sabitinin ve gevşek değişkenlerin eklenmesidir.

Bu değişiklik Lagrange çarpanlarına da yansır. Önceden α_i yalnız $\alpha_i \geq 0$ ile sınırlıydı. Şimdi bir üst sınır gelir:

$$0 \leq \alpha_i \leq C$$

Destek vektörü olan örneklerin α değeri yine sıfırdan büyüktür, ama C'den de küçük ya da ona eşittir. Hocanın özetiyle: C parametresi, kimin destek vektörü olacağını belirlerken α'ları sınırlamak için kullanılır.

> **Hocanın vurgusu:** Not alan varsa yazsın: Eşitsizliği gevşek değişkeni ekleyecek şekilde değiştiriyoruz (update ediyoruz) ve işin içine bir de gevşek değişkeni katıyoruz.

Hoca C'nin etkisini de söyledi: C büyüdükçe yanlış sınıflandırma oranı azalır. (Hocanın burada kullandığı "esnek" kelimesi için Ders Sonrası Notlar'a bakın.)

Hoca bölümü şu cümleyle bağladı: Ters taraftaki noktalar tekil noktalardır, genele uymayan noktalardır; sınıflandırma sınırı bunlara göre belirlenmemelidir. Hassasiyeti azaltmak için hiper düzlem yumuşatılır, bunun için gevşek değişken ve C parametresi eklenir.

12. Doğrusal Ayrılamayan Veri ve Kernel Fonksiyonları

Bu bölümde hocanın bağlantısı sık sık koptuğu için hoca bir süre yazamadan, slayt üzerinden konuşarak anlattı.

Slayttaki soldaki şekilde doğrusal olarak ayrılamayan bir örnek grubu vardır. A ve B sınıflarını bir doğruyla ayırmaya kalkarsanız başarısız, kötü bir sınıflandırma elde edersiniz.

SVM'in en faydalı özelliklerinden biri şudur: Bir dönüşüm (mapping) işlemiyle giriş örnekleri başka bir uzaya taşınır ve orada doğrusal olarak ayrılabilir hale getirilir. Bu dönüşüm kernel fonksiyonlarıyla yapılır. En çok kullanılanları:

- polinom (polynomial) kernel,
- Gaussian kernel,
- radyal tabanlı fonksiyon (radial basis function, RBF) kernel.

Mantık şudur: Karar fonksiyonunda eğitim örneği ile bilinmeyen örneğin iç çarpımını alıyorduk. Kernel kullanınca bu iç çarpımın yerine kernel fonksiyonunun çıkışı kullanılır.

Hocanın örneği (2. derece polinom kernel): Eğitim setindeki bir örneğin özellikleri $\mathbf{x} = (x_1, x_2)$, bilinmeyen örneğin özellikleri $\mathbf{y} = (y_1, y_2)$ olsun. Kernel fonksiyonunun çıkışı şudur:

$$K(\mathbf{x}, \mathbf{y}) = (1 + x_1 y_1 + x_2 y_2)^2 = (1 + \mathbf{x}^T \mathbf{y})^2$$

Böylece doğrusal olmayan durumdan doğrusal duruma geçilmiş olur.

> **Ek bilgi (derste anlatılmadı):** Bu kernel'ın neden "başka bir uzaya taşıma" olduğunu açmak mümkündür. $(1 + \mathbf{x}^T \mathbf{y})^2$ ifadesi açıldığında, iki boyutlu her örneği altı boyutlu $\phi(\mathbf{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1 x_2)$ vektörüne taşıyıp orada iç çarpım almakla aynı sonuç çıkar: $K(\mathbf{x}, \mathbf{y}) = \phi(\mathbf{x})^T \phi(\mathbf{y})$ (eşitlik python ile kontrol edildi). Kernel, bu yüksek boyutlu vektörleri hiç hesaplamadan aynı iç çarpımı verir. Buna kernel hilesi (kernel trick) denir.

Slayttaki genel kernel örnekleri (**x** eğitim örneği, **y** bilinmeyen örnek):

- Derecesi d olan polinom kernel: $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + 1)^d$. Hocanın örneğindeki gibi kare kullanırsanız $d = 2$ olur.
- Gaussian (RBF) kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\dfrac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$

Bağlantı düzelince hoca kalemle yazarak kernel'lı karar fonksiyonunu gösterdi:

$$f(\mathbf{z}) = \sum_{i \in SV} \alpha_i y_i \, K(\mathbf{x}_i, \mathbf{z}) + b$$

Burada y_i yine $+1$ ya da -1'dir. "Bakın şurada çarpı var" dedi: Önceki formüldeki iç çarpımın yerine, kernel'ın söylediği işlem neyse o yazılır. Örneğin Gaussian kernel seçildiyse, iç çarpımın yerine yukarıdaki üstel ifade gelir.

> **Hocanın vurgusu:** Bunu kendiniz yazmayacaksınız, hazır kütüphane kullanacaksınız. Ama kütüphaneye "Gaussian kernel kullan" dediğinizde yaptığı şey budur: Her destek vektörü ile test örneğine kernel'ı uygular, sonuçları toplar ve toplam sıfırdan büyükse bir sınıfa, küçükse diğer sınıfa atar.

13. Kernel ve C Seçimi: Doğrulama ile Model Seçmek

Gerçek bir problemde örneklerin doğrusal ayrılıp ayrılmadığını genellikle bilmeyiz. Özellik sayısı 3'ü geçtikten sonra veriyi bir grafiğe çizip bakma imkanımız da yoktur.

Bu yüzden hoca, geçen hafta gösterilen doğrulama (validation) mantığıyla çalışmayı önerdi:

1. Farklı C değerleri denenir. (C, aykırı örnekler yüzünden hiper düzlemin yerinin bozulduğu durumlar için kullanılıyordu.)
2. Farklı kernel'lar denenir: lineer, polinom, RBF, Gaussian.
3. Her kombinasyonla destek vektör makinesi eğitilir ve doğrulama verisinde başarısına bakılır.
4. En iyi sonucu veren model seçilir ve onunla devam edilir.

> **Hocanın vurgusu:** Lineer kernel da aslında iyi bir başarı verebilir. Hangi kernel'ın iyi olduğunu önceden bilemezsiniz; deneyerek seçin.

14. Çok Sınıflı SVM: One-Against-One

Hoca bu kısmı bilgisayarının şarjı bitmek üzereyken hızlı anlattı.

Buraya kadar anlatılan SVM ikili sınıflandırıcıdır. İkiden fazla sınıf olduğunda (multi-class recognition) ne yapılır? Bütün sınıfları birbirinden tek seferde ayıracak karar sınırları belirleyen yöntemler vardır, ama bunlar hem azdır hem de karmaşık ve yavaştır. Onun yerine çok kullanılan iki yaklaşım vardır.

Birinci yaklaşım one-against-one'dır (bire karşı bir). Hoca anlatmaya önce "one-against-all" diye başladı, sonra öğrencilere "şuna one-against-one der misiniz" diyerek adını düzeltti.

Eğitim aşaması: Diyelim ki 4 sınıf var: A, B, C, D. Her sınıf çifti için bir destek vektör makinesi hazırlanır: A-B, A-C, A-D, B-C, B-D, C-D.

> **Ek bilgi (derste anlatılmadı):** k sınıf için one-against-one'da hazırlanan SVM sayısı $\frac{k(k-1)}{2}$, one-against-all'da ise k'dir. Dersteki 4 sınıflı örnekte bu sayılar 6 ve 4'tür (python ile sayıldı). Sınıf sayısı arttıkça one-against-one'daki SVM sayısı karesel olarak büyür.

Bu SVM'ler eğitim aşamasında bir kez hazırlanır ve bir kenara konur.

> **Hocanın vurgusu:** Test aşamasında asla eğitimle ilgili bir iş yapmamalısınız, yoksa sisteminiz çok yavaş olur.

Test aşaması (hocanın örneği): Bilinmeyen bir $\mathbf{x}$ örneği gelir.

1. $\mathbf{x}$ önce A-B SVM'ine verilir. Bu SVM için çıkış pozitifse örnek A'ya, negatifse B'ye aittir. Sonuç $f(\mathbf{x}) > 0$ çıkıyor: A seçilir, B elenir.
2. $\mathbf{x}$ C-D SVM'ine verilir. Pozitifse C, negatifse D. Sonuç yine $f(\mathbf{x}) > 0$: C seçilir, D elenir.
3. Geriye A ve C kaldı. $\mathbf{x}$, hazırda bekleyen A-C SVM'ine verilir. Pozitifse A, negatifse C. Sonuç negatif çıkıyor.
4. Karar: $\mathbf{x}$ C sınıfına aittir.

Yani bilinmeyen örnek önce A-B ve C-D ile değerlendirildi, B ve D elendi, sonra A-C SVM'i son kararı verdi. Çok sınıflı sınıflandırmayı yapmanın yollarından biri budur.

15. Çok Sınıflı SVM: One-Against-All

İkinci yaklaşım one-against-all'dur (bire karşı hepsi). Hoca bunun bugün de çok kullanılan bir yaklaşım olduğunu vurguladı. Kullanım örnekleri:

- Yüz tespiti (face detection): yüz olan örnekler / yüz olmayan örnekler.
- Yaya tespiti (pedestrian detection): insan olan örnekler / insan olmayan örnekler.

Mantık şudur: Bir sınıfta tanımak istediğiniz örnekler, karşı sınıfta "her şey" vardır.

Eğitim aşaması: 4 sınıf (A, B, C, D) için 4 SVM hazırlanır:

- A / B, C, D karışık
- B / A, C, D karışık
- C / A, B, D karışık
- D / A, B, C karışık

Yaya örneğinde bir sınıfta insanlar, karşı sınıfta araçlar, evler, elektrik direkleri gibi insan olmayan bütün örnekler bulunur.

15.1 Soru-Cevap: Karşı sınıftaki dağılım

Soru: Karşı sınıftaki dağılım önemli mi? Mesela A sınıfında 50, karşı sınıfta 50 örnek olsa, o 50'nin içi de eşit dağılmalı mı?

Cevap: Öyle bir zorunluluk yok. Ama çok dengesiz yaparsanız başarısız olabilir: İki elektrik direği koyup 98 araç koyarsanız elektrik direğinin ne kadar etkili olacağını kestirmek zor. Zaten amaç elektrik direğini sınıflandırmak değil, örneğin insan olmadığını anlamaktır. Hoca bu durumu kendisinin çok denemediğini, ama karşı sınıfın olabildiğince iyi dağılmasını tavsiye ettiğini söyledi. İsteyen az örnek koyup sonra o az örnekli türden bir örnekle sistemi deneyerek etkisini görebilir.

Hoca bir güzelliğe de dikkat çekti: Karşı sınıfa hiç koymadığınız türden bir örnek test sırasında geldiğinde de sistem genellikle hata yapmaz. Yani karşı sınıfı iyi çeşitlendirirseniz sistem genellikle başarılı olur.

Test aşaması: Bilinmeyen **x** örneği dört SVM'in hepsine verilir. Her SVM'de tanınmak istenen sınıf pozitif, karşı sınıf negatif tarafı oluşturur. Kütüphaneler eğitim örneklerini tanıtırken bunu belirleme imkanı verir: Bir örneğin özelliklerini yazarken çıkışının $+1$ mi -1 mi olacağını siz söylersiniz. Kimin artı kimin eksi olduğunu siz belirlediğiniz için, bilinmeyen örneği hangi SVM pozitif sınıflandırdıysa örnek o sınıfa atanır.

Hocanın örneği: A / BCD için negatif, B / ACD için negatif, C / ABD için negatif, D / ABC için pozitif. Karar: Örnek D sınıfına aittir.

16. Hangisi Daha Başarılı?

Hoca öğrencilere sordu: "One-against-one mı daha başarılıdır, one-against-all mı? Korkmadan söyleyin, yanlış düşünmekte sakınca yok. Ben de başta başka bir şey düşünüyordum, düşündüğümden farklı çıkıyor."

- Bir öğrenci one-against-one dedi.
- Bir öğrenci one-against-all dedi ve gerekçesini söyledi: one-against-one'da sınıf sayısı arttıkça işlem çok daha karmaşıklaşıyor.

Bu sırada hocanın bağlantısı bir kez daha koptu. Geri dönünce cevabı verdi:

> **Hocanın vurgusu:** Hangi yöntemin ne zaman başarılı olacağı her zaman değişir; kullandığınız örneklerle ve problemle çok ilgilidir. Ama genellikle one-against-all, one-against-one'dan daha başarılı sonuç verir.

Hoca kendi deneyimini de anlattı: İlk kullanmaya başladığında one-against-one'ın daha başarılı olacağını düşünmüş. Çünkü orada A'ya karşı yalnız B'ye, A'ya karşı yalnız C'ye bakılıyor; sınıflar daha "saf" (pure). Ama gerçek uygulamalarda çoğunlukla one-against-all'un daha iyi sonuç verdiğini görmüş.

Hoca, bir şeyi tespit etmeye (detect) yönelik ikili sınıflandırma problemlerinde veri setlerinin genellikle bu şekilde hazırlandığını söyledi: yüz buldum, insan buldum gibi. Bu yalnız SVM için değil, başka modeller için de geçerlidir. Güncel bir örnek verdi: Bir mesajın hakaret içerip içermediğinin değerlendirilmesi. Bir sınıfta hakaret içeren mesajlar bulunur. Karşı tarafta güzel mesaj, üzgün mesaj gibi ayrı sınıflar kurulmaz; hepsi tek bir torbaya konur.

> **Hocanın vurgusu:** One-against-all, SVM dışındaki sınıflandırıcılarda da en çok kullanılan veri bölme yaklaşımıdır.

Dersin Toparlaması

Bu derste perceptron'un asıl probleminden yola çıktık: İki sınıfın arasına sonsuz sayıda karar sınırı çizilebilir ve perceptron bunlardan birini gelişigüzel seçer, ama test başarısını karar sınırının yeri belirler. Destek vektör makineleri bu sorunu, karar sınırını iki sınıfın birbirine en yakın örneklerine eşit uzaklığa koyarak çözer. Bu en yakın örnekler, yani ön cephedeki örnekler, destek vektörleridir; arkadaki örneklerin karar sınırına etkisi yoktur.

Margin'in genişliği $2/\|\mathbf{w}\|$'dir. Margin'i maksimize etmek, $\frac{1}{2}\|\mathbf{w}\|^2$'yi $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$ kısıtı altında minimize etmek demektir. Bu problem Lagrange çarpanlarıyla çözülür; ağırlık vektörü

$\mathbf{w} = \sum \alpha_i y_i \mathbf{x}_i$ biçimindedir ve yalnız $\alpha_i > 0$ olan destek vektörleri sınıflandırmaya katılır. Sınıra yakın ya da ters tarafa kaymış birkaç örnek margin'i gereksiz yere daraltmasın diye gevşek değişkenler ve C parametresiyle soft margin kullanılır; bu, α'lara C üst sınırını getirir.

Doğrusal ayrılamayan veride kernel fonksiyonları (polinom, Gaussian, RBF) örnekleri doğrusal ayrılabilecekleri bir uzaya taşır; karar fonksiyonundaki iç çarpımın yerine kernel yazılır. Hangi kernel'ın ve hangi C değerinin iyi olduğu önceden bilinmediği için doğrulama verisiyle denenerek seçilir. İkiden fazla sınıf için one-against-one (her sınıf çifti için bir SVM) ya da one-against-all (her sınıf için "bu sınıf / geri kalan hepsi" SVM'i) kullanılır; hocanın deneyimine göre gerçek uygulamalarda genellikle one-against-all daha başarılıdır.

Ders Sonrası Notlar

Ders sonrası not: Hoca bias için "hyperplane'in orijine oturmasını sağlar" dedi. Bias'ın işlevi tam tersidir: b (ya da w_0) sayesinde hiper düzlem orijinden geçmek zorunda kalmaz, orijinden kaydırılabilir. $b = 0$ olursa hiper düzlem orijinden geçer.

Ders sonrası not: Hoca destek vektörlerini tanımlarken önce "alfa i büyüktür 0 ise x'ler support vektörleri veriyor", hemen ardından "alfa i'nin 0 olduğu yerlerde x'ler bize support vektörleri veriyor" dedi. İkinci cümle dil sürçmesidir. Doğrusu: $\alpha_i > 0$ olan örnekler destek vektörüdür; $\alpha_i = 0$ olanlar değildir. Hoca biraz sonra doğru ifadeyi tekrarladı.

Ders sonrası not: Soft margin'de hoca örnek türlerini α üzerinden anlattı: "margin'deyse $\alpha_i = 0$, margin içindeyse $\alpha_i < 1$, hataysa $\alpha_i > 1$". Standart gösterimde bu üç tür gevşek değişkenle ayrılır: margin üzerinde ya da doğru tarafta dışarıda $\xi_i = 0$; margin içinde ama doğru tarafta $0 < \xi_i \leq 1$; yanlış sınıflanmış $\xi_i > 1$. Çarpanlar için ise $0 \leq \alpha_i \leq C$ geçerlidir: margin dışındaki örneklerde $\alpha_i = 0$, tam margin üzerindeki destek vektörlerinde $0 < \alpha_i < C$, margin içine girmiş ve yanlış sınıflanmış örneklerde $\alpha_i = C$ olur. Hoca da hemen ardından margin destek vektörleri için "α_i, C'den küçük" dedi.

Ders sonrası not: Hoca "C ne kadar büyürse o kadar daha esnek, yanlış sınıflandırma oranı daha az" dedi. Cümlenin ikinci yarısı eğitim verisi için doğrudur: Büyük C hatalara daha çok ceza verir ve eğitimde yanlış sınıflanan örnek sayısı azalır. Ama bu, margin'in daralması ve sınırın daha az esnek (daha katı) olması demektir; aşırı öğrenme riski artar. Küçük C ise daha geniş, daha esnek bir margin verir ve birkaç hatayı kabul eder.

Ders sonrası not: One-against-one'daki eleme (turnuva) biçimindeki test, hocanın derste anlattığı yoldur. Kütüphanelerde one-against-one genellikle bütün $\frac{k(k-1)}{2}$ SVM'i çalıştırır ve her SVM'in seçtiği sınıfa bir oy verir; en çok oy alan sınıf seçilir (voting).

Ders sonrası not: Hoca deneyimini anlatırken bir cümlede "gerçek uygulamalarda çoğunlukla one-against-one'ın daha iyi sonuç verdiğini gördüm" dedi. Hemen önceki cümleden ("genellikle one-against-all daha başarılı sonuç veriyor") ve başta tersini beklediğini söylemesinden, kastının one-against-all olduğu anlaşılıyor.

Ders sonrası not: Hoca SVM için "1992'de, Ruslar tarafından ortaya atılmış" dedi. Yöntem Vladimir Vapnik ve arkadaşlarına aittir. Vapnik'in istatistiksel öğrenme kuramı çalışmaları Sovyetler Birliği'nde başladı; maksimum margin'li SVM ve kernel kullanımı 1992'de Boser, Guyon ve Vapnik'in, soft margin ise 1995'te Cortes ve Vapnik'in çalışmasıyla yayımlandı. Bu çalışmalar ABD'de (AT&T Bell Laboratuvarları) yapıldı.

Ders sonrası not: Altyazıdaki bozulmalar: "Bayes" bias, "V" ve "v" ağırlık w, "hyperclaim/hyperplay" hyperplane, "Laurentian multiplier/laranjin şarpanları/larynge'nin makluları" Lagrange multipliers (Lagrange çarpanları), "L-Net" LeNet, "silek" slack, "sine fonksiyonu/sinir sonksiyonu" sign fonksiyonu, "göğüsün kernel/Gowsing kernel" Gaussian kernel, "Non-linearly-sacorable" non-linearly separable, "Monti Class/Mantiklas" multi-class, "One Against Sog/olun" one-against-all, "kreğe örneği/QERI" query (sorgu) örneği, "altlayır/alt-layır" outlier

demektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.